

דוח פרויקט רשתות תקשורת מחשבים - חלק 2: מערכת צ'אט

מגישים: אביטל יוחננוב, נאור אנידג'ר

בחלק זה של הפרויקט, המטרה הייתה לתכנן ולממש מערכת צ'אט מרובת משתמשים (Multi-Client Chat System) הפועלת על גבי פרוטוקול TCP.

המערכת מבוססת על ארכיטקטורת Client-Server (שרת-לקוח), כאשר השרת מנהל את החיבורים והניתוב, והלקוחות יכולים להתחבר, לראות סטטוס, ולהתכתב זה עם זה בהודעות פרטיות.

המימוש בוצע בשפת Python תוך שימוש בספריית socket לתקשורת ובספריית threading לטיפול במקביליות.

ארכיטקטורת המערכת

המערכת מורכבת משני רכיבים עיקריים:

השרת (server.py):

השרת מאזין לפורט קבוע (5000) וממתין לחיבורים נכנסים.

מכיוון שהשרת נדרש לטפל במספר לקוחות בו-זמנית (ולא לחסום לקוחות אחרים בזמן שהוא מטפל באחד), השתמשנו בגישת : Multi-Threading.

* עבור כל לקוח שמתחבר, השרת יוצר Thread ייעודי.

* Thread אחראי להאזין להודעות מאותו לקוח ספציפי ולטפל בהן (לנתב אותן ליעד או לבצע פקודות).

* השרת מחזיק מבני נתונים (Dictionaries) לניהול המשתמשים המחוברים (users) ולניהול השיחות הפעילות (peers). שימוש במנעול (threading.Lock) מבטיח סנכרון ומניעת התנגשויות גישה לזיכרון.

הלקוח (client.py / gui client.py):

הלקוח מתחבר לשרת ב-TCP. כדי לאפשר חוויית משתמש רציפה (שליחת הודעות וקבלת הודעות במקביל), גם הלקוח משתמש בשני Threads:

1. Main Thread: אחראי על ממשק המשתמש.

2. Listening Thread: רץ ברקע ומאזין להודעות נכנסות מהשרת כדי להציג אותן למשתמש מיד כשהן מגיעות.

פרוטוקול התקשורת

הגדרנו פרוטוקול טקסטואלי פשוט לניהול השיחה. כל פקודה מסתיימת בתו ירידת שורה.(\n)

-התחברות (שליחת שם בלבד):

ברגע החיבור, הלקוח שולח את שם המשתמש שלו לרישום.

-יצירת שיחה:(CONNECT <username>)

בקשה לחיבור לשיחה פרטית מול משתמש אחר.

-שליחת הודעה:(MSG <text>)

שליחת הודעת טקסט למשתמש שאיתו מתקיימת השיחה.

-ניתוק:(DISCONNECT)

סיום השיחה הנוכחית (מבלי להתנתק מהשרת).

ממשק משתמש גרפי (GUI)

בנוסף לממשק הטקסטואלי פיתחנו ממשק משתמש גרפי מלא באמצעות ספריית Tkinter.

הממשק כולל:

*מסך התחברות (Login) מעוצב.

*חיווי ויזואלי על סטטוס החיבור (Online/Offline)

*הפרדה צבעונית בין הודעות יוצאות (שלי) להודעות נכנסות (של החבר).

*עיצוב מודרני (Dark Mode) לנוחות השימוש.

הרצת המערכת ותוצאות

להלן צילומי מסך המדגימים את פעולת המערכת:

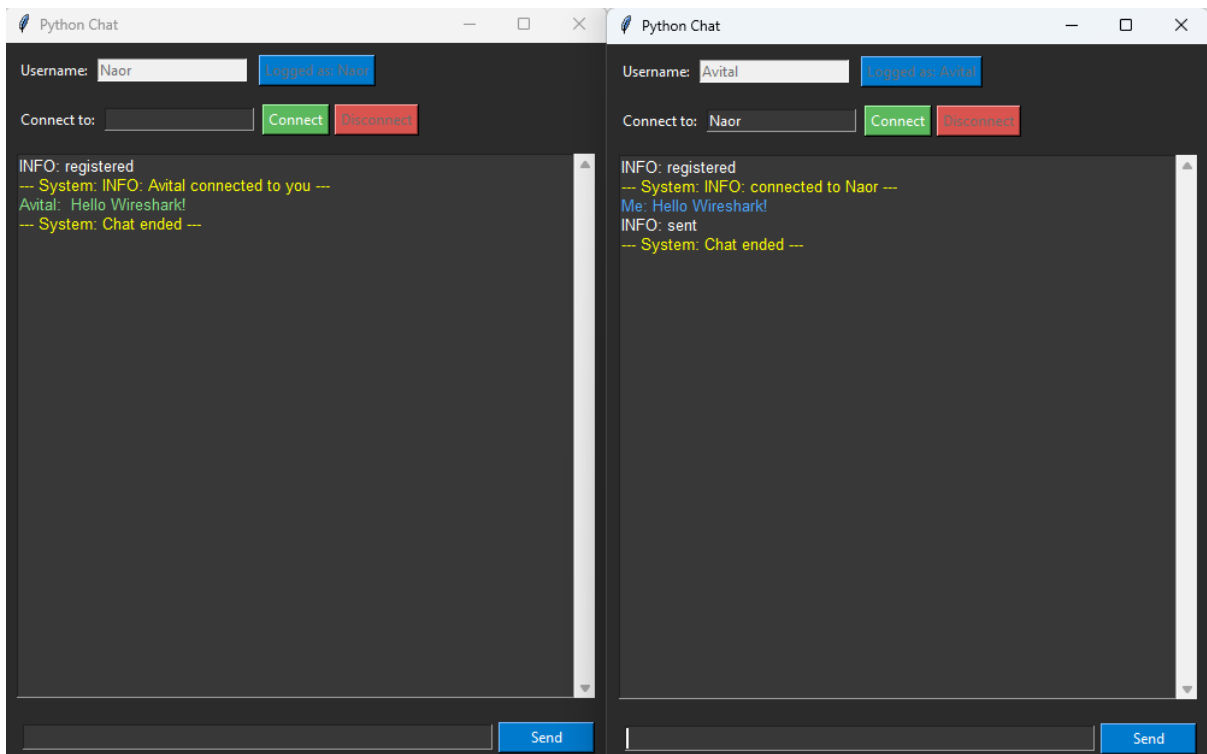
א. מסך השרת: (Server Console)

ניתן לראות את הלוגים של השרת המעידים על משתמשים שהתחברו ונוצרו להם Threads.

```
PS C:\Users\Avital\Desktop\לנסיון\תוכנת תחבורה\project part 2> python server.py
Server is running... waiting for clients
Client connected: ('127.0.0.1', 51677)
User registered: Avital
Client connected: ('127.0.0.1', 56996)
User registered: Naor
█
```

ב. ממשק הלקוח (GUI) ושיחה פעילה:

בתמונה זו ניתן לראות שני לקוחות מחוברים ומנהלים שיחה ביניהם. ניתן לראות שהודעות עוברות מצד לצד בזמן אמת.



ניתוח תעבורה ב-Wireshark

כדי לוודא שהמערכת אכן משתמשת בפרוטוקול TCP ושולחת את המידע ברשת, ביצענו האזנה לממשק ה- Loopback בפורט 5000.

בצילום המסך ניתן לראות את תעבורת ה-TCP (הדגלים PSH, ACK) ואת תוכן הפרוטוקול שלנו עובר כ-Plain text.

The screenshot shows the Wireshark interface with a packet capture on the 'Adapter for loopback traffic capture' interface. The packet list shows a series of TCP packets, including a SYN, ACK, and several data packets. The packet details pane shows the selected packet (Seq=0, Win=65535, Len=0, MSS=65495, WS=256, SACK_PERM [SYN], ACK) and its contents, which are the raw bytes of the data being transmitted. The packet bytes pane shows the hex and ASCII representation of the data, including the text 'ket, 57 bytes on wire (456 bits), 57 bytes captured (456 bits) on interface \Device\NPF_{...} Loopback, id 0 < Null/Loopback < Internet Protocol Version 4, Src: 127.0.0.1, Dst: 127.0.0.1 < Transmission Control Protocol, Src Port: 51677, Dst Port: 5000, Seq: 8, Ack: 18, Len: 13 < Data (13 bytes) <

The screenshot shows the Wireshark interface with a packet capture on the 'Adapter for loopback traffic capture' interface. The packet list shows a series of TCP packets, including a SYN, ACK, and several data packets. The packet details pane shows the selected packet (Seq=0, Win=65535, Len=0, MSS=65495, WS=256, SACK_PERM [SYN], ACK) and its contents, which are the raw bytes of the data being transmitted. The packet bytes pane shows the hex and ASCII representation of the data, including the text 'ket, 57 bytes on wire (456 bits), 57 bytes captured (456 bits) on interface \Device\NPF_{...} Loopback, id 0 < Null/Loopback < Internet Protocol Version 4, Src: 127.0.0.1, Dst: 127.0.0.1 < Transmission Control Protocol, Src Port: 51677, Dst Port: 5000, Seq: 8, Ack: 18, Len: 13 < Data (13 bytes) <

לסיכום

בפרויקט זה יישמנו מערכת תקשורת מלאה מקצה לקצה. התמודדנו עם אתגרי תכנות מקבילי (Threading), תכנון פרוטוקול אפליקציה, ועבודה עם Sockets.